

A FSM do Controlador/Sequenciador do SAP-1

Este documento explica a máquina de estados (FSM) que comanda o SAP-1, acompanha os dois diagramas gerados (`fsm_diagram.*` e `fsm_exec_diagram.*`) e mostra um **exemplo rodando passo a passo**.

Código de referência: `controller_fsm.v` (versão FSM clássica) e `controller_sequencer.v` (versão contador de anel — equivalente).

1. O que a FSM faz

O SAP-1 não executa uma instrução "de uma vez". Cada instrução é quebrada em **micropassos**, um por estado de tempo (*T-state*). A FSM é só um contador que percorre esses estados em ordem e, **em cada estado**, liga os sinais de controle certos (a *palavra de controle* CON, de 12 bits).

São 6 estados de trabalho — **T1 a T6** — mais um estado **IDLE** de partida. Cada instrução leva exatamente 6 ciclos:

Fase	Estados	Faz o quê	Depende do opcode?
Busca (<i>fetch</i>)	T1, T2, T3	lê a instrução da RAM e coloca no IR	Não (igual p/ todas)
Execução (<i>execute</i>)	T4, T5, T6	faz o trabalho da instrução	Sim

É por isso que existem dois diagramas: `fsm_diagram` mostra o **anel de estados** (a "pista" que a FSM corre); `fsm_exec_diagram` mostra **o que cada instrução faz** dentro de T4-T6.

2. Os 3 blocos da FSM (formato clássico)

Em `controller_fsm.v` a máquina está escrita nos três processos canônicos:

- Registrador de estado** — guarda o estado atual (`state`). Avança na **borda de descida** do clock (`negedge clk`); o reset assíncrono (`~CLR=0`) leva para **IDLE** .
- Lógica de próximo estado** (combinacional) — a sequência fixa `IDLE → T1 → T2 → T3 → T4 → T5 → T6 → T1 → ...` . Quando o processador está parado (`run = 0`), a FSM **segura o estado** (congela).
- Lógica de saída** (combinacional, Moore) — gera a palavra CON a partir do estado (e, em T4-T6, também do opcode).

Por que a descida do clock?

A FSM troca de estado na **descida**, e os registradores do *datapath* (PC, A, B, IR, ...) carregam na **subida**. Assim, quando chega a subida, a palavra de controle daquele T-state **já está estável** — sem condição de corrida.

Por que o estado IDLE?

Ele "absorve a fase do reset": não importa em que ponto do clock você solte o `~CLR`, a máquina começa limpa e o primeiro `negedge` útil leva a T1.

Como o HLT para tudo (sem desligar o clock)

O sinal `halted` é travado em **T4 quando o opcode é HLT**. Ele faz `run = 0`, que é um **clock-enable**: a FSM para de avançar e a palavra de controle vira IDLE → nenhum registrador carrega → tudo congela. O clock continua oscilando (boa prática de FPGA — nada de *gating* de clock).

3. A palavra de controle (CON)

```
CON = { Cp, Ep, ~Lm, ~CE, ~Li, ~Ei, ~La, Ea, Su, Eu, ~Lb, ~Lo }
bit:  11 10  9  8  7  6  5  4  3  2  1  0
```

- Sinais **sem til** (Cp, Ep, Ea, Su, Eu): ativos em **ALTO** (1 = ativo).
- Sinais **com til** (~Lm, ~CE, ~Li, ~Ei, ~La, ~Lb, ~Lo): ativos em **BAIXO** (0 = ativo).

Significado rápido:

Sinal	Ação quando ativo
Ep	PC coloca seu valor no barramento
~Lm	MAR carrega do barramento
Cp	PC incrementa (+1)
~CE	RAM coloca o dado no barramento
~Li	IR carrega do barramento
~Ei	IR coloca o operando (endereço) no barramento
~La	Acumulador A carrega do barramento
Ea	A coloca seu valor no barramento
~Lb	Registrador B carrega do barramento
Eu	ALU coloca o resultado no barramento
Su	ALU subtrai (em vez de somar)
~Lo	Registrador de saída carrega do barramento

4. O ciclo de busca (T1-T3) — igual para toda instrução

Estado	Sinais ativos	Transferência	Em palavras
T1	Ep, ~Lm	MAR ← PC	"o endereço da próxima instrução vai para o MAR"
T2	Cp	PC ← PC + 1	"o PC já aponta para a instrução seguinte"
T3	~CE, ~Li	IR ← RAM[MAR]	"a instrução é lida e guardada no IR"

No fim de T3 o opcode já está no IR — então T4-T6 podem decidir o que fazer.

5. O ciclo de execução (T4-T6) — depende do opcode

Resumo do que cada instrução faz (ver detalhes em fsm_exec_diagram):

Instr.	Opcode	T4	T5	T6
LDA	0000	~Lm, ~Ei MAR←IR.end	~CE, ~La A←RAM	—
ADD	0001	~Lm, ~Ei MAR←IR.end	~CE, ~Lb B←RAM	~La, Eu A←A+B
SUB	0010	~Lm, ~Ei MAR←IR.end	~CE, ~Lb B←RAM	~La, Eu, Su A←A-B
OUT	1110	Ea, ~Lo OUT←A	—	—
HLT	1111	trava run←0	congelado	congelado

Onde um passo não faz nada (—), a FSM emite a palavra IDLE, mas **ainda gasta o ciclo** (todas as instruções duram 6 estados).

6. Exemplo rodando passo a passo

Vamos seguir as **3 primeiras instruções** do programa que está na RAM (ram_16x8.v), que calcula $3^3 = 27$. Endereço 12 contém o dado **3**.

```
end 0: LDA 12 ; A <- 3
end 1: ADD 12 ; A <- 3 + 3 = 6
end 2: ADD 12 ; A <- 6 + 3 = 9 (= 3^2)
```

Estado inicial após o reset: PC = 0, A = 0, B = 0.

Instrução 1 — LDA 12 (em PC=0)

T	Sinais	O que acontece	Estado depois
T1	Ep, ~Lm	MAR ← PC(0)	MAR=0
T2	Cp	PC ← 1	PC=1
T3	~CE, ~Li	IR ← RAM[0] = LDA 12	IR=0000_1100
T4	~Lm, ~Ei	MAR ← IR.end = 12	MAR=12

T	Sinais	O que acontece	Estado depois
T5	$\sim CE, \sim La$	$A \leftarrow RAM[12] = 3$	A=3
T6	(idle)	nada (LDA termina em 5 passos úteis)	—

→ Fim da instrução: **A = 3**, PC = 1.

Instrução 2 — **ADD 12** (em PC=1)

T	Sinais	O que acontece	Estado depois
T1	$Ep, \sim Lm$	$MAR \leftarrow PC(1)$	MAR=1
T2	Cp	$PC \leftarrow 2$	PC=2
T3	$\sim CE, \sim Li$	$IR \leftarrow RAM[1] = \text{ADD } 12$	IR=0001_1100
T4	$\sim Lm, \sim Ei$	$MAR \leftarrow 12$	MAR=12
T5	$\sim CE, \sim Lb$	$B \leftarrow RAM[12] = 3$	B=3
T6	$\sim La, Eu$	$A \leftarrow A + B = 3 + 3$	A=6

→ Fim da instrução: **A = 6**, PC = 2.

Instrução 3 — **ADD 12** (em PC=2)

T	Sinais	O que acontece	Estado depois
T1	$Ep, \sim Lm$	$MAR \leftarrow PC(2)$	MAR=2
T2	Cp	$PC \leftarrow 3$	PC=3
T3	$\sim CE, \sim Li$	$IR \leftarrow RAM[2] = \text{ADD } 12$	IR=0001_1100
T4	$\sim Lm, \sim Ei$	$MAR \leftarrow 12$	MAR=12
T5	$\sim CE, \sim Lb$	$B \leftarrow RAM[12] = 3$	B=3
T6	$\sim La, Eu$	$A \leftarrow A + B = 6 + 3$	A=9

→ Fim da instrução: **A = 9 = 3²**, PC = 3.

A próxima instrução do programa é **OUT** (PC=3), que copia A para o visor — mostrando **09** antes de seguir para a segunda metade do cálculo (chegando a $27 = 3^3$).

Resumo do exemplo

Após a instrução	A (decimal)	Significado
LDA 12	3	carregou o dado 3
ADD 12	6	$3 + 3$
ADD 12	9	$3 + 3 + 3 = 3^2$

Cada uma dessas instruções foi **6 estados da FSM** (T1-T6). O contador de anel simplesmente repete T1...T6 para cada instrução até encontrar o **HLT**, que congela tudo.

7. Como reproduzir os diagramas

```
python3 fsm_diagram.py      # anel de estados -> fsm_diagram.{png,pdf,svg}  
python3 fsm_exec_diagram.py # execução por instr -> fsm_exec_diagram.{png,pdf,svg}
```

Os `.pdf` / `.svg` são vetoriais — ideais para colar no relatório sem perder qualidade.